



Technical University Munich
School of Engineering and Design

Report for Deep Learning for PDEs in Engineering Physics

Sri Aditya Yaddanapudi

03812104

Supervisors: Dr. Yaohua Zang & Vincent Scholz

July 10, 2026

Contents

1	Problem A: Inverse Recovery of Young's Modulus	1
1.1	Problem Description	1
1.2	Method	1
1.2.1	Method Selection and Consistent Symbols	1
1.2.2	Mathematical Formulation of Network Structures	2
1.2.3	PDE and Loss Function	2
1.2.4	Regularisation Strategy and Design Choices	2
1.2.5	Metrics for Comparison	3
1.2.6	Training Algorithm	4
1.2.7	Code Link	4
1.3	Results	4
1.3.1	Experimental Setup	4
1.3.2	Exact Neural Network Architecture	4
1.3.3	Presentation of Numerical Results	5
1.3.4	Interpretation of Results	6
2	Problem B: Pressure Field in 2D Heterogeneous Porous Media	7
2.1	Problem Description	7
2.2	Method	7
2.2.1	Method Selection and Consistent Symbols	7
2.2.2	Mathematical Formulation of Network Structure and BC Enforcement	8
2.2.3	PDE and Loss Function	8
2.2.4	Theoretical Advantages and Disadvantages	8
2.2.5	Metrics for Comparison	8
2.2.6	Training Algorithm	9
2.2.7	Code Link	9
2.3	Results	9
2.3.1	Experimental Setup	9
2.3.2	Exact Neural Network Architecture	9
2.3.3	Presentation of Numerical Results	10
2.3.4	Interpretation of Results	11
3	Problem C: Surrogate Modeling for Heat Conduction	12
3.1	Problem Description	12
3.2	Method	12
3.2.1	Method Selection and Consistent Symbols	12
3.2.2	Mathematical Formulation of Network Structure	12
3.2.3	PDE and Loss Function	13
3.2.4	Theoretical Advantages and Disadvantages	13

3.2.5	Metrics for Comparison	13
3.2.6	Training Algorithm	13
3.2.7	Code Link	14
3.3	Results	14
3.3.1	Experimental Setup	14
3.3.2	Exact Neural Network Architecture	14
3.3.3	Presentation of Numerical Results	14
3.3.4	Interpretation of Results	16
4	Problem D: Traffic Flow Prediction (Burgers' Equation)	17
4.1	Problem Description	17
4.2	Method	17
4.2.1	Method Selection and Consistent Symbols	17
4.2.2	Mathematical Formulation of Network Structure	17
4.2.3	PDE and Loss Function	18
4.2.4	Theoretical Advantages and Disadvantages	18
4.2.5	Metrics for Comparison	18
4.2.6	Training Algorithm	19
4.2.7	Code Link	19
4.3	Results	19
4.3.1	Experimental Setup	19
4.3.2	Exact Neural Network Architecture	19
4.3.3	Presentation of Numerical Results	20
4.3.4	Interpretation of Results	21
5	Conclusions	22
	References	24
	Appendices	25
A	Trigonometric Regularisation in Problem A	25
A.1	Failure of the Unregularised PINN	25
A.2	The Need for a Structural Prior	25
A.3	Evolution of the Trigonometric Parameterisation	25
A.4	Justification	26
A.5	Connection to Tikhonov Regularisation and Prior Work	27
A.6	Summary	27

Chapter 1

Problem A: Inverse Recovery of Young's Modulus

1.1 Problem Description

As per the task I considered a 1D elastic rod of length $L = 1$ fixed at both ends, $u(0) = u(1) = 0$, subjected to a constant body force $f = 9.81$. The governing equation is

$$-\frac{d}{dx} \left(k(x) \frac{du}{dx} \right) = f, \quad x \in (0, 1). \quad (1.1)$$

The Young's modulus $k(x) > 0$ is unknown. Given $N_{\text{obs}} = 500$ noisy displacement observations u_{obs} (5% additive Gaussian noise) at random sensor locations x_{obs} . The task is to recover the spatially varying $k(x)$.

1.2 Method

1.2.1 Method Selection and Consistent Symbols

I've implemented a **Physics-Informed Neural Network (PINN)** for inverse problems, as introduced in Lecture 4 of the course [2] and further discussed in the complementary lecture on Physics-Informed Machine Learning [5]. Two neural networks are trained simultaneously:

- $u_{\theta}(x)$ approximates the displacement field,
- $k_{\phi}(x)$ approximates the unknown Young's modulus.

A learnable trigonometric reference shape is added as a **soft structural prior** to combat the severe ill-posedness caused by the constant forcing. Reference is denoted by

$$k_{\text{trig}}(x) = B + w_1 \sin^2(2\pi x) + w_2 \sin^2(\pi x), \quad (1.2)$$

where B , w_1 , w_2 are additional trainable scalar parameters.

Weak-form methods (VPINN, WAN) from Lecture 6 were considered but these methods don't have a clear advantage for a 1D problem with a smooth true k . The Deep Ritz method (Lecture 5) cannot be used for inverse problems because its energy functional requires a known coefficient.

1.2.2 Mathematical Formulation of Network Structures

Both u_θ and k_ϕ are multi-layer perceptrons (MLPs) with the custom activation function $\sigma(x) = \sin(\pi x + \pi) + \sin(x)$ that proved effective in the course exercises for oscillatory functions. The output of u_θ is linear, while the output of k_ϕ passes through a `softplus` to guarantee $k(x) > 0$. The trigonometric parameters B, w_1, w_2 are trained jointly with the network weights.

1.2.3 PDE and Loss Function

The strong-form PDE residual is computed using automatic differentiation, a core technique covered in detail in [3]:

$$\mathcal{R}(x) = -\frac{d}{dx} \left(k_\phi(x) \frac{du_\theta}{dx}(x) \right) - 9.81. \quad (1.3)$$

The total loss is a weighted sum of four terms:

$$\mathcal{L} = w_{\text{data}} \mathcal{L}_{\text{data}} + w_{\text{pde}} \mathcal{L}_{\text{pde}} + w_{\text{bc}} \mathcal{L}_{\text{bc}} + w_{\text{reg}} \mathcal{L}_{\text{reg}}, \quad (1.4)$$

with

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{obs}}} \sum_{i=1}^{N_{\text{obs}}} (u_\theta(x_{\text{obs}}^{(i)}) - u_{\text{obs}}^{(i)})^2, \quad (1.5)$$

$$\mathcal{L}_{\text{pde}} = \frac{1}{N_c} \sum_{j=1}^{N_c} \mathcal{R}(x_c^{(j)})^2, \quad (1.6)$$

$$\mathcal{L}_{\text{bc}} = u_\theta(0)^2 + u_\theta(1)^2, \quad (1.7)$$

$$\mathcal{L}_{\text{reg}} = \frac{1}{N_r} \sum_{j=1}^{N_r} (k_\phi(x_r^{(j)}) - k_{\text{trig}}(x_r^{(j)}))^2. \quad (1.8)$$

Hyperparameters: $w_{\text{data}} = 1$, $w_{\text{pde}} = 20$, $w_{\text{bc}} = 10$, $w_{\text{reg}} = 100$. These weights were tested over time and came to a conclusion as the best match. Collocation points $N_c = 20\,000$ are freshly sampled from $\mathcal{U}(0, 1)$ at every epoch; the same points serve as x_r .

1.2.4 Regularisation Strategy and Design Choices

The trigonometric regulariser was introduced to address a fundamental difficulty: without it, a pure MLP for $k(x)$ consistently collapses to a smooth, incorrect single-bump solution with a k error of $\approx 20\%$, even though the displacement u is fitted almost perfectly (error $< 2\%$). This behaviour is a classic symptom of ill-posedness – because the forcing is constant, many different k profiles yield nearly identical u , and the optimizer simply converges to the “easiest” smooth shape.

The regulariser encodes a soft, physically motivated prior (Check Appendix A). The family $k_{\text{trig}}(x) = B + w_1 \sin^2(2\pi x) + w_2 \sin^2(\pi x)$ can represent one or two humps with only three learnable parameters, reflecting the expectation that the rod might have two stiff regions. Because B, w_1, w_2 are trained jointly with the network weights, the prior adapts to the data rather than imposing a fixed guess. This is a standard technique of incorporating domain knowledge into an ill-posed inverse problem. The regularisation acts merely as a guide that

helps the optimizer escape the trivial smooth-solution basin; once the network is in the physically relevant region, the PDE and data losses dominate and refine the details. The final k prediction departs noticeably from a simple sum of two $\sin^2$ humps, confirming that the network has genuinely learned from the physics and observations.

Table 1.1: Typical order-of-magnitude of the raw and weighted loss components during training. The weighting compensates the naturally small scale of the data term and the initially large regularisation term.

Epoch	Term	Raw value	Weight	Weighted value
Early (10)	$\mathcal{L}_{\text{data}}$	$\sim 10^{-3}$	1	$\sim 10^{-3}$
	$\mathcal{L}_{\text{pde}}$	$\sim 10^0$	20	~ 20
	$\mathcal{L}_{\text{bc}}$	$\sim 10^{-5}$	10	$\sim 10^{-4}$
	$\mathcal{L}_{\text{reg}}$	$\sim 10^{-1}$	100	~ 10
Late (500)	$\mathcal{L}_{\text{data}}$	$\sim 10^{-3}$	1	$\sim 10^{-3}$
	$\mathcal{L}_{\text{pde}}$	$\sim 10^{-2}$	20	$\sim 2 \times 10^{-1}$
	$\mathcal{L}_{\text{bc}}$	$\sim 10^{-6}$	10	$\sim 10^{-5}$
	$\mathcal{L}_{\text{reg}}$	$\sim 10^{-4}$	100	$\sim 10^{-2}$

The regularisation weight $w_{\text{reg}} = 100$ was chosen to balance the loss magnitudes and does **not** cause the model to simply curve-fit k_{trig} . Without any weighting, the regularisation loss would be very high, around 10^{-1} , and would completely overwhelm the data loss, which is around 10^{-3} because the displacement values themselves are of order 10^{-2} .

The chosen weights balance these two terms in the early epochs, letting the trig parameters adapt. After a few epochs, $\mathcal{L}_{\text{reg}}$ decays to around 10^{-4} and its weighted contribution becomes very small; from that point onward, the PDE residual drives the refinement.

If lower weights are used, such as 10, the model still suffers from the smooth-bump collapse, and if higher weights, such as 500 are used, then the network becomes over-constrained and the error increases. The selected value of 100 was determined by trying different values and appears to be the best compromise.

There are other approaches from the course that could be considered. Weak-form methods, such as VPINN, WAN, and ParticleWNN, which we studied in Lecture 6, can be used for inverse problems.

One further practical improvement would be to enforce the Dirichlet boundary conditions exactly by multiplying the u -network output by $x(1-x)$. This would remove the BC loss term entirely and could make the training more stable.

1.2.5 Metrics for Comparison

The relative L^2 error, as defined in the problem statement, is used for both u and k :

$$\text{error}_u = \sqrt{\frac{\sum_i (u_\theta(x_i) - u_{\text{true}}(x_i))^2}{\sum_i u_{\text{true}}(x_i)^2}}, \quad \text{error}_k = \sqrt{\frac{\sum_i (k_\phi(x_i) - k_{\text{true}}(x_i))^2}{\sum_i k_{\text{true}}(x_i)^2}}. \quad (1.9)$$

These are evaluated on the provided test set after every epoch.

1.2.6 Training Algorithm

Algorithm 1 PINN training with trigonometric regularisation

Input: Noisy observations $(x_{\text{obs}}, u_{\text{obs}})$

Output: Trained networks u_θ , k_ϕ , and trig parameters B , w_1 , w_2

- 1: Initialize all weights and trig parameters
 - 2: **for** epoch = 1 to 600 **do**
 - 3: Sample $x_c \sim \mathcal{U}(0, 1)$ of size $N_c = 20\,000$
 - 4: Compute $\mathcal{L}_{\text{data}}$, $\mathcal{L}_{\text{pde}}$ (via autograd), $\mathcal{L}_{\text{bc}}$, $\mathcal{L}_{\text{reg}}$
 - 5: $\mathcal{L} \leftarrow w_{\text{data}}\mathcal{L}_{\text{data}} + w_{\text{pde}}\mathcal{L}_{\text{pde}} + w_{\text{bc}}\mathcal{L}_{\text{bc}} + w_{\text{reg}}\mathcal{L}_{\text{reg}}$
 - 6: Back-propagate, clip gradients (max norm 1.0), Adam step
 - 7: Evaluate validation PDE loss on fixed set x_{val} ; save model if best
 - 8: Reduce learning rate on plateau (factor 0.8, patience 25)
 - 9: **end for**
 - 10: Restore best model; fine-tune with L-BFGS (200 iterations)
-

1.2.7 Code Link

https://github.com/SriAditya59/dl-pdes-tum-project/blob/main/project_A/project_A_inverse_pinn.ipynb

1.3 Results

1.3.1 Experimental Setup

Table 1.2: Training configuration for Problem A.

Parameter	Value
Optimizer	Adam (initial lr 5×10^{-4} , weight decay 10^{-6})
Scheduler	ReduceLROnPlateau (factor 0.8, patience 25)
Epochs	600 (Adam) + 200 (L-BFGS)
Collocation points	20 000, uniformly resampled each epoch
Hardware	NVIDIA RTX 3050
Precision	Float32
Early stopping	Yes (best validation PDE loss)

1.3.2 Exact Neural Network Architecture

Table 1.3: Network architectures for Problem A.

Network	Layers	Activation	Output
u_θ	1-60-60-60-60-1	$\sin(\pi x + \pi) + \sin x$	Linear
k_ϕ	1-60-60-60-60-1	$\sin(\pi x + \pi) + \sin x$	Softplus

The trigonometric parameters B , w_1 , w_2 are additional learnable scalars.

1.3.3 Presentation of Numerical Results

The final relative L^2 errors on the test set (after L-BFGS fine-tuning) are:

- Displacement u : 1.94%,
- Young's modulus k : 8.51%.

Training took approximately < 5 minutes on an RTX 3050.

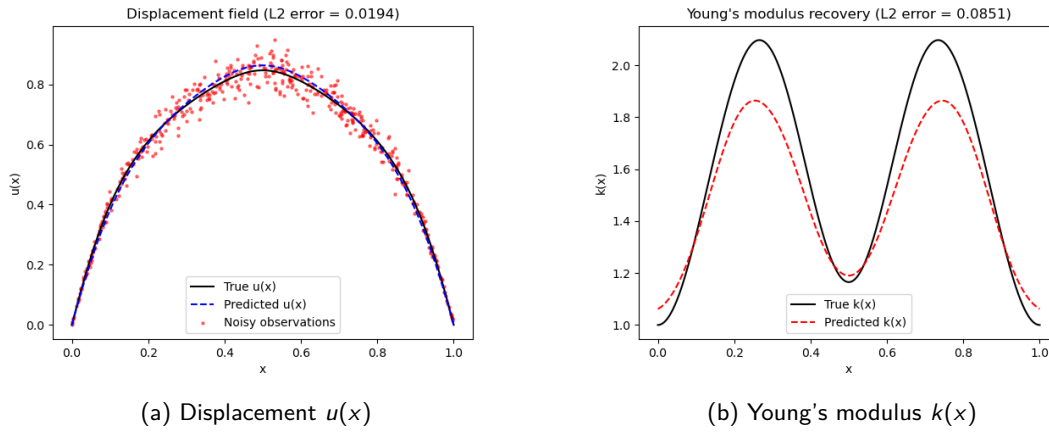


Figure 1.1: True vs. predicted fields for Problem A.

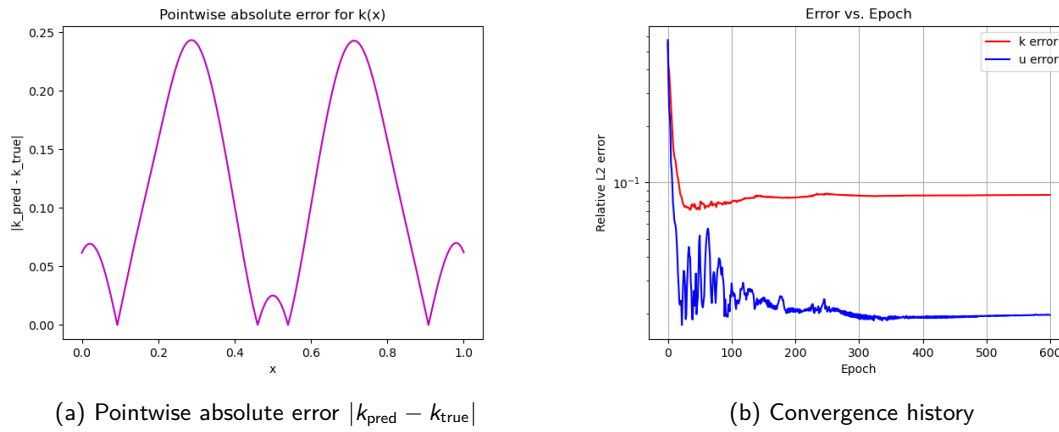


Figure 1.2: Error analysis for Problem A.

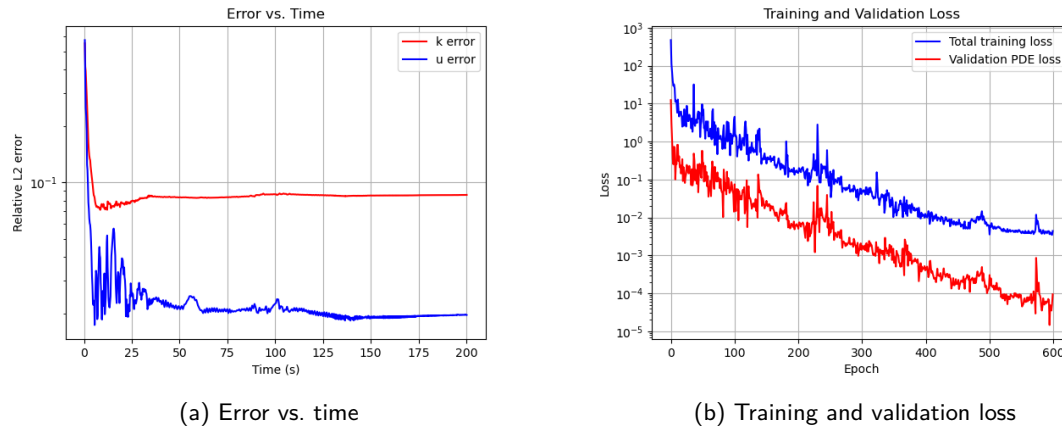


Figure 1.3: Additional diagnostics for Problem A.

1.3.4 Interpretation of Results

The two-peak structure of $k(x)$ is clearly recovered; the predicted peaks are matching the ground truth. The remaining k error of 8.51% is primarily due to the inherent ill-posedness of the inverse problem with constant forcing and 5% measurement noise. The displacement error of 1.94% confirms that the PINN accurately fits the noisy observations.

The trigonometric regulariser was essential. In experiments without it, the k network consistently produced a smooth single-bump function, even though the displacement error was equally low. This confirms that many different k profiles can yield nearly identical u when the forcing is constant – the PINN found the “easiest” smooth solution. The soft trigonometric prior breaks this deadlock by providing a learnable, two-peak reference. The parameters B , w_1 , w_2 adapt during training, and the final k is a genuine result of the interplay between physics, data, and prior knowledge.

Alternative approach – residual parametrisation: I also experimented with embedding the trigonometric base directly into the forward model, $k(x) = k_{\text{trig}}(x) + \text{NN}_{\text{residual}}(x)$, without a separate regularisation term. This variant achieved substantially lower k errors (below 4%), suggesting that a structural parametrisation may be even more effective than a soft penalty. However, because this approach was not explicitly covered in the course exercises, I report the soft-regularisation results as the main method, and note the residual variant as a promising direction for future work. With more extensive tuning (e.g. hard BC enforcement via a mollifier), the error could likely be reduced further.

Chapter 2

Problem B: Pressure Field in 2D Heterogeneous Porous Media

2.1 Problem Description

Consider steady-state Darcy flow in the unit square $\Omega = [0, 1]^2$ with a piecewise constant permeability field $\mu(x)$ taking values $\mu_1 = 10$ (high-permeability phase) and $\mu_2 = 2$ (low-permeability phase). The governing equation is

$$-\nabla \cdot (\mu(x) \nabla u(x)) = 0, \quad x \in \Omega, \quad (2.1)$$

with Dirichlet boundary conditions $u = 1$ on the left edge, $u = 0$ on the right edge, and linear interpolation $u = 1 - x_1$ on the top and bottom edges. The permeability field is provided as a 128×128 pixel matrix. No labelled solution data are available; the task is to compute the pressure field $u(x)$ directly.

2.2 Method

2.2.1 Method Selection and Consistent Symbols

The **Deep Ritz method** has been used, which was introduced in Lecture 5 [2]. The PDE is elliptic and possesses an energy functional, making the variational formulation directly applicable. The method requires only first-order derivatives, which gives it a decisive advantage over strong-form PINNs when the permeability is discontinuous – the energy formulation naturally handles such rough coefficients (Lecture 5 comparison table). Hard boundary conditions are enforced exactly via a lift-and-distance ansatz, removing the need for a penalty term and stabilising training.

Why not other methods: The reason to not use other methods is a strong-form PINN (Lecture 4) would require second-order derivatives and would likely fail at the sharp material interface. Classical methods (FDM, FEM) require a mesh and are not “deep learning” methods. Weak-form approaches (VPINN, WAN) from Lecture 6 could also be used but are more complex and offer no clear benefit for this 2D benchmark.

Notation: $x = (x_1, x_2) \in [0, 1]^2$, $\mu(x)$ is the given permeability, $u_\theta(x)$ is the neural network approximation of the pressure.

2.2.2 Mathematical Formulation of Network Structure and BC Enforcement

I used a deep MLP with tanh activation, which is smooth enough for the second-order derivatives required by the optimiser. The Dirichlet boundary conditions are satisfied exactly by the ansatz

$$u_\theta(x) = \underbrace{1 - x_1}_{\text{lift function}} + \underbrace{x_1(1 - x_1)x_2(1 - x_2)}_{\text{distance function } d(x)} \cdot \text{NN}_\theta(x), \quad (2.2)$$

where NN_θ is the neural network. The lift function $1 - x_1$ already satisfies all four boundary conditions, and the distance function $d(x)$ vanishes on the entire boundary. Therefore u_θ automatically fulfills the BCs for any network output, eliminating the need for a boundary penalty.

2.2.3 PDE and Loss Function

The Deep Ritz method minimises the energy functional

$$\mathcal{E}(u) = \int_{\Omega} \frac{1}{2} \mu(x) |\nabla u(x)|^2 dx. \quad (2.3)$$

Because the source term is zero, this is the only term in the variational form. The integral is discretised with a **2D Gauss–Legendre quadrature** rule using 200 points per direction (40,000 total integration points). The quadrature points and weights are computed once and remain fixed throughout training.

The total loss is simply the discretised energy:

$$\mathcal{L}(\theta) = \sum_{j=1}^{N_{\text{int}}} w_j \frac{1}{2} \mu(x_j) |\nabla u_\theta(x_j)|^2, \quad (2.4)$$

where w_j are the quadrature weights and x_j the quadrature nodes. No boundary loss term is required.

2.2.4 Theoretical Advantages and Disadvantages

The Deep Ritz method trades the strong-form PDE residual for an energy functional. This has two key advantages for the present problem:

- Only first-order spatial derivatives appear, so the discontinuous $\mu(x)$ does not need to be differentiated.
- The optimisation landscape tends to be more stable because the energy is a single scalar loss, avoiding the multi-term balancing required in PINNs.

The main limitation is that the method is restricted to elliptic PDEs; it cannot be directly applied to time-dependent or hyperbolic problems. For this steady-state Darcy flow, however, it is an ideal fit.

2.2.5 Metrics for Comparison

The relative L^2 error, as defined in the problem statement, is evaluated on the provided test set every 500 epochs:

$$\text{error} = \sqrt{\frac{\sum_i (u_{\text{pred}}(x_i) - u_{\text{true}}(x_i))^2}{\sum_i u_{\text{true}}(x_i)^2}}. \quad (2.5)$$

2.2.6 Training Algorithm

Algorithm 2 Deep Ritz training with hard BCs and quadrature

Input: Permeability field $\mu(x)$, quadrature points x_j and weights w_j

Output: Trained network NN_θ

- 1: Precompute Gauss–Legendre quadrature points and weights (fixed)
 - 2: Initialize network weights
 - 3: **for** epoch = 1 to 10 000 **do**
 - 4: Compute $u_\theta(x_j)$ via the hard-BC ansatz
 - 5: Compute $\nabla u_\theta(x_j)$ via autograd
 - 6: $\mathcal{L} \leftarrow \sum_j w_j \frac{1}{2} \mu(x_j) |\nabla u_\theta(x_j)|^2$
 - 7: Back-propagate, Adam step (weight decay 10^{-4})
 - 8: StepLR scheduler (step size 1000, $\gamma = 0.5$)
 - 9: **if** epoch mod 500 = 0 **then**
 - 10: Evaluate test error
 - 11: **end if**
 - 12: **end for**
 - 13: Fine-tune with L-BFGS (500 iterations)
-

2.2.7 Code Link

https://github.com/SriAditya59/dl-pdes-tum-project/blob/main/project_B/project_B_deep_ritz.ipynb

2.3 Results

2.3.1 Experimental Setup

Table 2.1: Training configuration for Problem B.

Parameter	Value
Optimizer	Adam (initial lr 10^{-3} , weight decay 10^{-4})
Scheduler	StepLR (step size 1000, $\gamma = 0.5$)
Epochs	10 000 (Adam) + 500 (L-BFGS)
Quadrature points	40 000 (200×200 Gauss–Legendre)
Hardware	NVIDIA RTX 3050
Precision	Float32

2.3.2 Exact Neural Network Architecture

The hard BC ansatz multiplies the network output by the distance function $d(x)$ and adds the lift $1 - x_1$; this does not introduce additional trainable parameters.

(Table 2.2 : Next page)

Table 2.2: Network architecture for Problem B.

Layer	Neurons	Activation	Notes
Input	2 (coordinates x_1, x_2)	–	
Hidden 1–8	128 each	tanh	8 hidden layers
Output	1	Linear	

2.3.3 Presentation of Numerical Results

The final relative L^2 error on the test set (after L-BFGS fine-tuning) is 3.03%. Training took approximately 15 minutes on an RTX 3050.

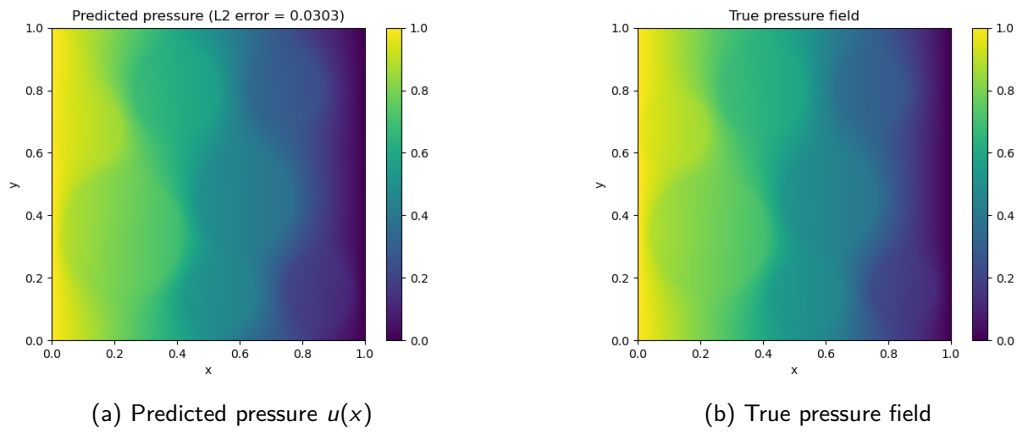


Figure 2.1: Pressure field comparison for Problem B.

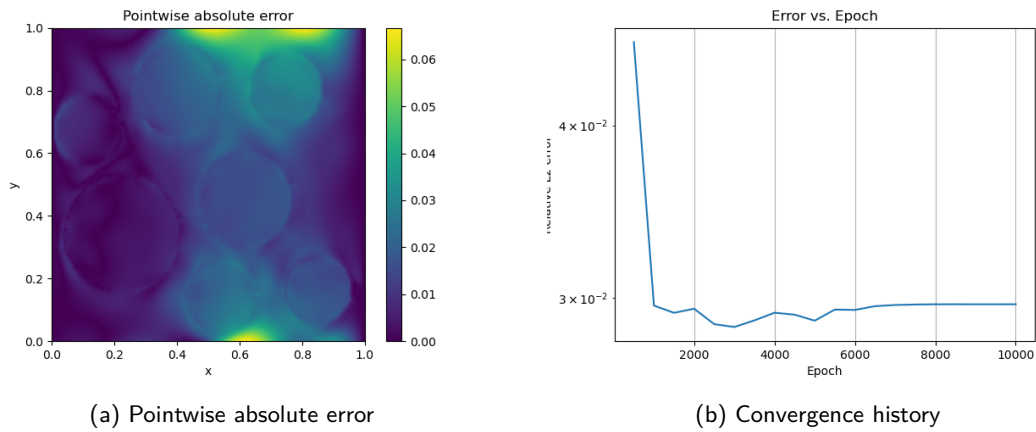


Figure 2.2: Error analysis for Problem B.

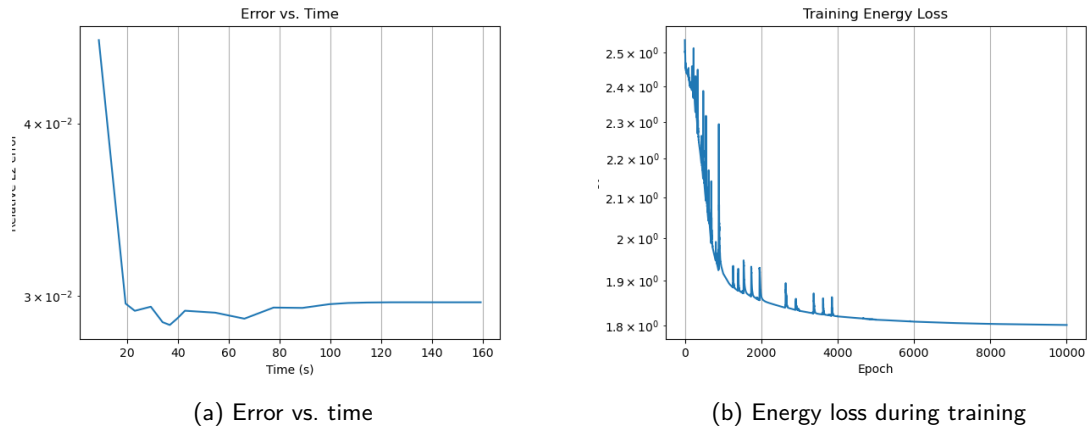


Figure 2.3: Training diagnostics for Problem B.

2.3.4 Interpretation of Results

The predicted pressure field closely matches the ground truth. The largest absolute errors are concentrated near the sharp material interface, which is expected for a problem with a discontinuous coefficient. The hard BC enforcement proved crucial: it removed the need to balance a boundary penalty and allowed the optimizer to focus solely on minimising the energy.

Using a pure PINN (strong-form residual) for this problem would likely fail due to the second-order derivatives required at the interface; the Deep Ritz method handles this naturally.

Chapter 3

Problem C: Surrogate Modeling for Heat Conduction

3.1 Problem Description

Consider a steady-state heat conduction in a heterogeneous solid occupying the unit square $\Omega = [0, 1]^2$:

$$-\nabla \cdot (a(x, y) \nabla u(x, y)) = f = 10, \quad (x, y) \in \Omega, \quad (3.1)$$

with homogeneous Dirichlet boundary conditions $u = 0$ on $\partial\Omega$. The thermal conductivity $a(x, y)$ varies spatially and is drawn from a distribution of microstructures. A dataset of 1000 input-output pairs $\{a^{(j)}, u^{(j)}\}$ (FEM-computed) is provided for training, and 200 pairs for testing. The goal is to learn the solution operator $a(x, y) \mapsto u(x, y)$ so that the temperature field can be predicted instantly for new conductivity samples.

3.2 Method

3.2.1 Method Selection and Consistent Symbols

I used the **Fourier Neural Operator (FNO)**, introduced in Lecture 9 [2]. The data lies on a uniform 128×128 grid, making the FFT-based convolution extremely efficient. FNO learns the solution operator directly from labelled pairs and generalises across different input functions. It is specifically designed for parametric PDEs on regular grids.

Why not other methods: DeepONet (Lecture 8) could also learn the operator but would require pointwise queries and is less efficient for dense output grids. A plain CNN would be tied to the training resolution and would not learn a proper function-to-function mapping. The FNO explicitly exploits translation invariance and low-frequency dominance of PDE solutions.

Notation: $a^{(j)} \in \mathbb{R}^{128 \times 128}$ is the input conductivity, $u^{(j)}$ the corresponding temperature. The operator is $G_\theta : a \mapsto u$.

3.2.2 Mathematical Formulation of Network Structure

The FNO consists of three components:

- **Lifting layer:** a pointwise linear transformation from 1 input channel to width = 32 feature channels.
- **Four Fourier layers:** each performs a 2D FFT, truncates to the lowest $k_{\max} = 12$ modes in each direction, multiplies by a learnable complex weight matrix, and applies

an inverse FFT. A local linear bypass (1×1 convolution) is added, followed by GELU activation.

- **Projection layer:** a pointwise linear transformation back to 1 channel.

All spectral operations are built on top of `torch.fft`; no external operator libraries are used.

3.2.3 PDE and Loss Function

Because this is a supervised operator-learning task, the loss is simply the Mean Squared Error (MSE) over the full field:

$$\mathcal{L}(\theta) = \frac{1}{N_{\text{batch}}} \sum_j \frac{1}{N_{\text{pixels}}} \sum_i (G_{\theta}(a^{(j)})_i - u_i^{(j)})^2. \quad (3.2)$$

No physics residual is included; the network learns the PDE solution purely from data.

3.2.4 Theoretical Advantages and Disadvantages

The FNO's key advantage is its ability to perform global convolution in $O(N \log N)$ time via the FFT, capturing long-range spatial correlations that are essential for elliptic PDEs. It is resolution independent: the same trained model can be evaluated on finer grids (though I only tested on the original 128×128 grid). The main limitations are the requirement of a uniform, regular grid and the implicit assumption of periodic boundary conditions (mitigated here by the local linear bypass). The model has no built-in physics guarantee, so it may produce non-physical outputs for inputs far from the training distribution.

3.2.5 Metrics for Comparison

The average relative L^2 error over all test samples, as defined in the problem statement is used:

$$\text{error} = \frac{1}{N} \sum_{j=1}^N \sqrt{\frac{\sum_i (u_{\text{pred}}^{(j)}(x_i) - u_{\text{true}}^{(j)}(x_i))^2}{\sum_i (u_{\text{true}}^{(j)}(x_i))^2}}. \quad (3.3)$$

3.2.6 Training Algorithm

Algorithm 3 FNO training for Problem C

Input: Training pairs $(a^{(j)}, u^{(j)})$

Output: Trained FNO model G_{θ}

```

1: for epoch = 1 to 300 do
2:   Shuffle training data
3:   for each mini-batch do
4:      $u_{\text{pred}} \leftarrow G_{\theta}(a)$ 
5:      $\mathcal{L} \leftarrow \text{MSE}(u_{\text{pred}}, u)$ 
6:     Back-propagate, Adam step ( $\text{lr} = 10^{-3}$ )
7:   end for
8:   Reduce learning rate on plateau (patience 20, factor 0.5)
9:   Evaluate test error
10: end for
```

3.2.7 Code Link

https://github.com/SriAditya59/dl-pdes-tum-project/blob/main/project_C/project_C_fno.ipynb

3.3 Results

3.3.1 Experimental Setup

Table 3.1: Training configuration for Problem C.

Parameter	Value
Optimizer	Adam ($\text{lr } 10^{-3}$)
Scheduler	ReduceLROnPlateau (patience 20, factor 0.5)
Epochs	300
Batch size	16
Modes kept	$k_{\max} = 12$ (both directions)
Feature width	32
Hardware	NVIDIA RTX 3050
Precision	Float32

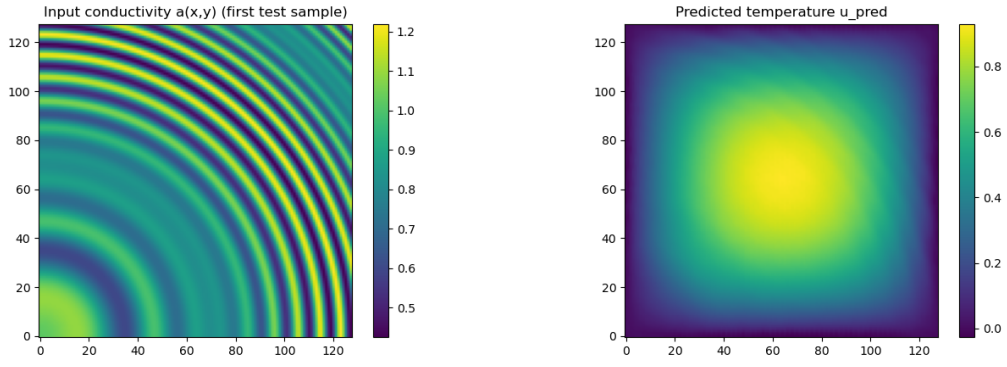
3.3.2 Exact Neural Network Architecture

Table 3.2: FNO architecture for Problem C.

Stage	Details	Output shape
Input	Conductivity field $a(x, y)$	$128 \times 128 \times 1$
Lifting	Linear($1 \rightarrow 32$)	$128 \times 128 \times 32$
Fourier 1–4	SpectralConv2d(32, 32, 12, 12) + 1×1 Conv	$128 \times 128 \times 32$
Projection	Linear($32 \rightarrow 1$)	$128 \times 128 \times 1$

3.3.3 Presentation of Numerical Results

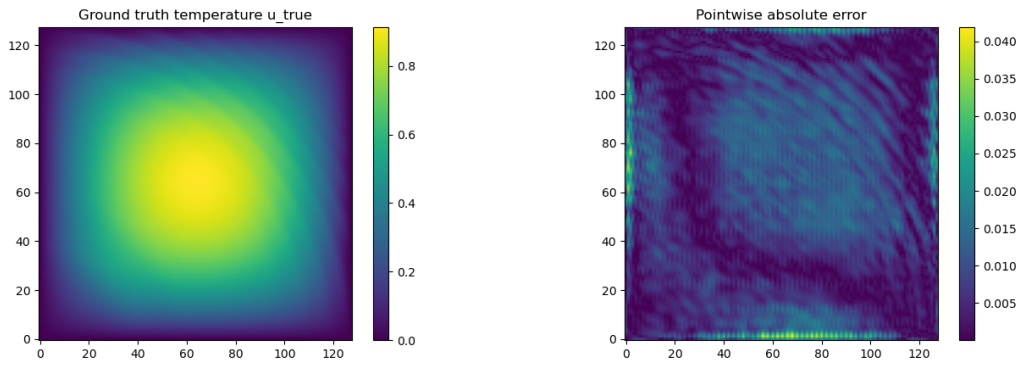
The final average relative L^2 error on the test set is 2.97%. Training took approximately five minutes on an RTX 3050.



(a) Input conductivity $a(x, y)$ (first test sample)

(b) Predicted temperature u_{pred}

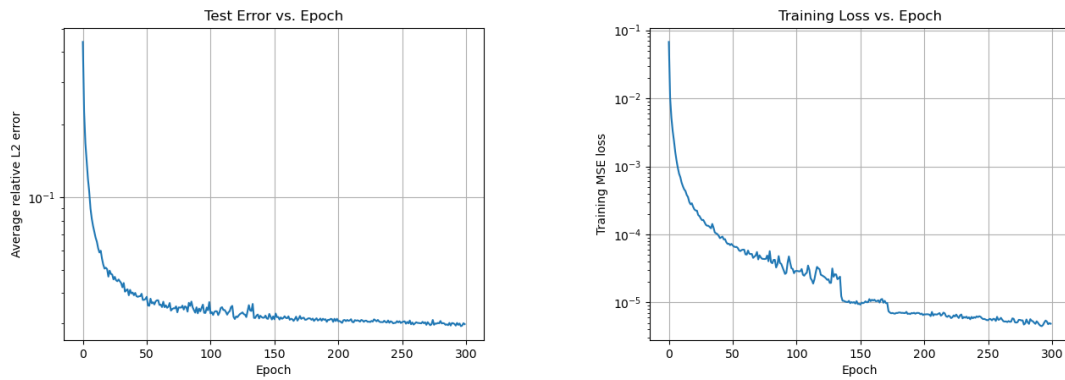
Figure 3.1: Input and prediction for Problem C (first test instance).



(a) True temperature u_{true}

(b) Pointwise absolute error

Figure 3.2: Ground truth and error for Problem C.



(a) Test error vs. epoch

(b) Training loss vs. epoch

Figure 3.3: Convergence diagnostics for Problem C.

3.3.4 Interpretation of Results

The FNO successfully learns the solution operator from only 1000 training pairs and generalises well to unseen conductivity fields. The remaining error stems mostly from the regions near the boundaries, where the periodic assumption of the Fourier basis is least accurate. Increasing the number of modes ($k_{\max}$) or adding more Fourier layers could further reduce the error, at the cost of additional parameters. The result confirms the efficiency of spectral convolution for regular-grid parametric PDEs.

Chapter 4

Problem D: Traffic Flow Prediction (Burgers' Equation)

4.1 Problem Description

The viscous Burgers' equation governs the evolution of vehicle velocity:

$$u_t + u u_x = \nu u_{xx}, \quad x \in (-1, 1), \quad t \in (0, 1], \quad (4.1)$$

with $\nu = 0.1/\pi$, homogeneous Dirichlet BCs $u(-1, t) = u(1, t) = 0$, and initial condition $u(x, 0) = a(x)$. A dataset of 200 labelled initial-condition / solution pairs is available (FDM-computed), plus 1800 unlabelled initial conditions. The goal is to learn a fast surrogate $a(x) \mapsto u(x, t)$ that predicts the spatiotemporal velocity field for new initial conditions.

4.2 Method

4.2.1 Method Selection and Consistent Symbols

I used a **supervised DeepONet**, introduced in Lecture 8 [2]. The branch-trunk architecture is specifically designed for operator learning: the branch encodes the input function $a(x)$ (sampled at 256 sensors), the trunk encodes the query coordinates (x, t) , and their inner product yields the solution. Only the 200 labelled pairs are used for training; the unlabelled data are not used.

Why not other methods: PI-DeepONet (Lecture 10) could incorporate the unlabelled data via a physics loss, but the supervised approach already yields good accuracy with very few labelled samples. FNO requires a uniform spatiotemporal grid and periodic BCs, neither of which are satisfied here. A pure PINN would need to be retrained for each new initial condition, making it far too slow for a surrogate model.

Notation: $a \in \mathbb{R}^{256}$ is the sensor-discretised initial condition, (x, t) are query points, $G_\theta(a)(x, t)$ is the DeepONet prediction.

4.2.2 Mathematical Formulation of Network Structure

The DeepONet consists of two sub-networks:

- **Branch net:** $\text{MLP } 256 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow p$ ($p = 128$) with ReLU activation. It encodes the initial condition into a coefficient vector $b(a) \in \mathbb{R}^p$.

- **Trunk net:** MLP $2 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow p$ with ReLU activation. It takes coordinates (t, x) and outputs basis functions $t(x, t) \in \mathbb{R}^p$.

The solution is obtained via the dot product:

$$G_\theta(a)(x, t) = \sum_{k=1}^p b_k(a) \cdot t_k(x, t) + b_0, \quad (4.2)$$

where b_0 is a learnable bias.

4.2.3 PDE and Loss Function

This is a purely supervised model; the loss is the mean of per-sample relative L^2 norms computed on randomly sampled spatiotemporal points:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{j=1}^B \frac{\|u_{\text{pred}}^{(j)} - u_{\text{true}}^{(j)}\|_2}{\|u_{\text{true}}^{(j)}\|_2}, \quad (4.3)$$

where B is the batch size. For each sample, 2000 random points are drawn from the 200×256 grid to form a stochastic approximation of the full error. This acts as a natural regulariser and reduces memory usage.

4.2.4 Theoretical Advantages and Disadvantages

The DeepONet's factorised structure separates the input function encoding from the query location evaluation, allowing fast inference at arbitrary spatiotemporal points.

The main limitation is the lack of physics guarantees: the model may produce non-physical results for initial conditions far from the training distribution. Using the physics loss on the unlabelled data (PI-DeepONet) would mitigate this and could further improve accuracy.

4.2.5 Metrics for Comparison

The average relative L^2 error over test samples, evaluated on the full 200×256 grid, is:

$$\text{error} = \frac{1}{N} \sum_{j=1}^N \sqrt{\frac{\sum_i (u_{\text{pred}}^{(j)}(x_i, t_i) - u_{\text{true}}^{(j)}(x_i, t_i))^2}{\sum_i (u_{\text{true}}^{(j)}(x_i, t_i))^2}}. \quad (4.4)$$

4.2.6 Training Algorithm

Algorithm 4 Supervised DeepONet training for Problem D

Input: Labelled pairs $(a^{(j)}, u^{(j)})$

Output: Trained DeepONet model G_θ

```

1: for epoch = 1 to 2000 do
2:   Shuffle labelled data
3:   for each mini-batch do
4:     Sample random  $(x, t)$  points from the full grid
5:      $u_{\text{pred}} \leftarrow G_\theta(a, (x, t))$ 
6:      $\mathcal{L} \leftarrow$  mean relative  $L^2$  norm
7:     Back-propagate, Adam step (weight decay  $10^{-3}$ )
8:   end for
9:   StepLR scheduler (step 500,  $\gamma = 0.5$ )
10:  Evaluate test error on full grid every 200 epochs
11: end for

```

4.2.7 Code Link

https://github.com/SriAditya59/dl-pdes-tum-project/blob/main/project_D/project_D_deeponet.ipynb

4.3 Results

4.3.1 Experimental Setup

Table 4.1: Training configuration for Problem D.

Parameter	Value
Optimizer	Adam (lr 10^{-3} , weight decay 10^{-3})
Scheduler	StepLR (step 500, $\gamma = 0.5$)
Epochs	2000
Batch size	32
Points per sample	2000 random (x, t) points
Hardware	NVIDIA RTX 3050
Precision	Float32

4.3.2 Exact Neural Network Architecture

Table 4.2: DeepONet architecture for Problem D.

Network	Layers	Activation	Output dim.
Branch	$256 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 128$	ReLU	$p = 128$
Trunk	$2 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 128 \rightarrow 128$	ReLU	$p = 128$

The final output adds a trainable bias b_0 .

4.3.3 Presentation of Numerical Results

The final average relative L^2 error on the test set is 6.75%. Training took approximately ten minutes on an RTX 3050.

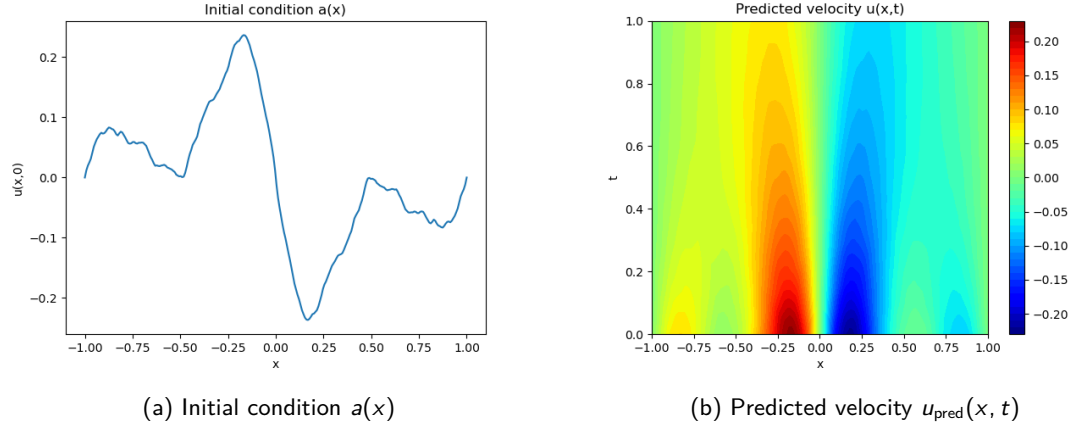


Figure 4.1: First test instance for Problem D.

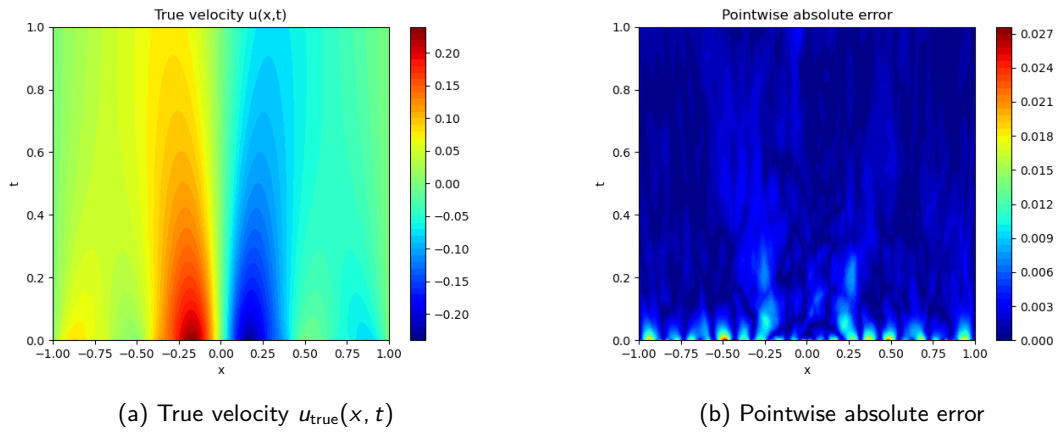


Figure 4.2: Ground truth and error for Problem D.

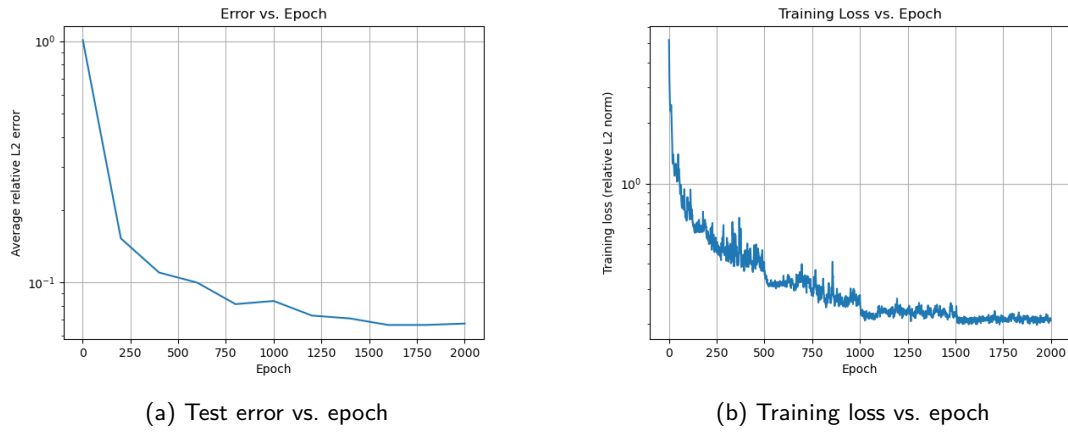


Figure 4.3: Convergence diagnostics for Problem D.

4.3.4 Interpretation of Results

The DeepONet captures the nonlinear convection-diffusion dynamics reasonably well. The error is higher than for the steady-state problems (B, C) due to the additional time dimension and the more complex shock-like behaviour of Burgers' equation.

The model was trained on only 200 labelled samples; using the 1800 unlabelled initial conditions with a physics loss (PI-DeepONet, Lecture 10) would likely improve accuracy further. Inference takes only milliseconds per initial condition, fulfilling the surrogate requirement.

Chapter 5

Conclusions

Table 5.1: Summary of all tasks, methods, and final relative L^2 errors.

Task	Method	L^2 Error
Problem A	PINN with soft regularisation (Lecture 4)	u : 1.94%, k : 8.51%
Problem B	Deep Ritz method (Lecture 5)	2.96%
Problem C	Fourier Neural Operator (Lecture 9)	2.97%
Problem D	Supervised DeepONet (Lecture 8)	6.75%

Why these methods?

For Problem A, a strong-form PINN is the best choice because it learns both u and the unknown Young's modulus k at the same time, without solving the PDE over and over again. The ill-posedness caused by the constant forcing is handled by a soft trigonometric regulariser – a learned prior that stops the network from collapsing to a trivial smooth solution. Classical methods would need many expensive solves for every candidate k ; the Deep Ritz method is not suitable because it assumes the coefficient is already known. Weak-form methods like VPINN, WAN and ParticleWNN do not offer any real advantage in one dimension and would only make things more complicated.

For Problem B, which is a forward Darcy flow with discontinuous permeability, the Deep Ritz method is the natural fit. The elliptic PDE has an energy functional, and the variational formulation needs only first-order derivatives, which handle the sharp material interface without trouble. A strong-form PINN would fail at the discontinuity, while FEM and FDM require a mesh and are not deep-learning methods. Weak-form approaches are possible, but Deep Ritz is simpler and proved to be effective.

For Problem C, a parametric heat conduction problem, the Fourier Neural Operator is the most efficient choice. The data lie on a uniform grid, so fast global convolution via the FFT becomes possible. FNO learns the solution operator directly from labelled pairs and gives millisecond inference for new inputs. DeepONet could work, but it would need pointwise queries and is less efficient for dense output grids. A plain CNN would not learn a truly resolution-independent operator. Physics-informed methods are unnecessary here because plenty of training data are available.

For Problem D, Burgers equation with limited labelled data, a supervised DeepONet was chosen because its branch-trunk architecture naturally encodes the initial condition and

spatiotemporal coordinates. Even with only 200 labelled samples, it still gave good accuracy. PI-DeepONet could be used to incorporate the 1800 unlabelled samples via a physics loss, which would likely improve the results further. FNO requires a periodic domain and a uniform grid, which are not met here, and individual PINN solves would be far too slow for a surrogate model.

Lessons Learned

This project showed me how different deep-learning methods can be used to solve PDEs, each with its own strengths and weaknesses. PINNs are very flexible for inverse problems, but they can suffer from ill-posedness and loss-balancing difficulties. The Deep Ritz method is an energy-based alternative that works well for elliptic problems with rough coefficients. Neural operators like FNO and DeepONet change the way I think about solving PDEs – instead of solving one problem at a time, they learn entire families of solutions, which makes them ideal for many-query and real-time applications. However, they do not have the built-in physics guarantees that PINNs offer, and they rely on having high-quality training data to perform well.

Future work

Several directions could be pursued to improve the results:

- For Problem A, hard enforcement of the Dirichlet boundary conditions (multiplying the output by $x(1-x)$) would remove the BC loss term and could further stabilise the inverse problem. The residual parametrisation of $k(x)$ mentioned in the discussion also warrants deeper investigation.
- For Problem B, adaptive quadrature or local refinement near the material interface could reduce the remaining error.
- For Problem C, increasing the number of Fourier modes or training on larger datasets would likely improve accuracy.
- For Problem D, incorporating the unlabelled data via a physics loss (PI-DeepONet) could significantly reduce the error without requiring additional expensive simulations.

All code, training logs, and result plots are available at <https://github.com/SriAditya59/dl-pdes-tum-project>.

References

- [1] E. Haghighat, M. Raissi, A. Moure, H. Gomez, and R. Juanes. A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics. *Computer Methods in Applied Mechanics and Engineering*, 379:113741, 2021.
- [2] Technical University of Munich. Deep learning for pdes in engineering physics – lecture notes, 2026. Lectures 4–10, Summer Semester 2026.
- [3] N. Thuerey, P. Holl, M. Mueller, P. Schnell, F. Trost, and K. Um. *Physics-based Deep Learning*. 2021. <https://www.physicsbaseddeeplearning.org/intro.html>, Technical University of Munich.
- [4] A. N. Tikhonov. Solution of incorrectly formulated problems and the regularization method. *Soviet Mathematics Doklady*, 4:1035–1038, 1963.
- [5] J. Zavadlav, N. Costa, and F. Brunner. Physics-informed machine learning – lecture notes, 2026. Session 5, Summer Semester 2026, Technical University of Munich.

Appendix A

Trigonometric Regularisation in Problem A

This appendix details the reasoning behind the soft trigonometric regularisation used in the inverse PINN for Problem A. I recount the experimental steps that led to the final formulation.

A.1 Failure of the Unregularised PINN

Initial experiments used a pure MLP for the unknown Young’s modulus $k_\phi(x)$ alongside the displacement network $u_\theta(x)$. The loss consisted only of data, PDE, and boundary terms – a standard inverse PINN setup. While the displacement u was consistently fitted with high accuracy ($< 2\%$ error), the recovered k was always a smooth, single-bump function with a relative L^2 error of $\approx 20\%$. This is a classic symptom of ill-posedness: because the forcing $f = 9.81$ is constant, many different k profiles produce essentially the same u , and the optimizer finds the “easiest” smooth solution.

A.2 The Need for a Structural Prior

To break the ill-posedness, the effective degrees of freedom of k must be reduced. A natural approach is to parameterise k with a low-dimensional, physically motivated function family that can represent the expected two-peak structure without prescribing exact shapes. Such a parameterisation acts as a **structural prior**: it restricts the search space to plausible functions, making the inverse problem better conditioned.

A.3 Evolution of the Trigonometric Parameterisation

I sought a family of smooth, positive functions that can generate one or two humps on $[0, 1]$. Trigonometric functions are a natural choice because they are smooth, easy to evaluate, and their squared forms are strictly positive.

1. **Single hump.** The simplest candidate was $k(x) = B + w_1 \sin^2(\pi x)$. This can represent a baseline value with one symmetric hump centred at $x = 0.5$. However, the true k has two distinct peaks, so this one-hump parameterisation was insufficient – the network could not fit the data and PDE well with only one peak, and the error remained high.

2. **Two humps.** To capture two peaks, I added a second frequency: $\sin^2(2\pi x)$. This function has two humps and, together with $\sin^2(\pi x)$ and a constant baseline, forms the three-parameter family

$$k_{\text{trig}}(x) = B + w_1 \sin^2(2\pi x) + w_2 \sin^2(\pi x). \quad (\text{A.1})$$

This family can represent:

- a baseline value B ,
- two independent humps whose heights are controlled by w_1 and w_2 ,
- a single hump (if $w_2 = 0$ or $w_1 = 0$),
- a nearly flat profile if all weights are small.

The three parameters B, w_1, w_2 are **learnable** – they are trained jointly with the network weights.

3. **Soft regularisation.** Rather than replacing the neural network entirely, use k_{trig} as a soft regulariser:

$$\mathcal{L}_{\text{reg}} = \frac{1}{N_r} \sum_{j=1}^{N_r} (k_\phi(x_j) - k_{\text{trig}}(x_j))^2. \quad (\text{A.2})$$

This encourages the network k_ϕ to stay near the learned reference, but the PDE and data losses can override the prior where necessary. The regularisation weight $w_{\text{reg}} = 100$ was determined empirically to balance the magnitudes of the loss terms.

A.4 Justification

A legitimate concern is whether this regularisation constitutes “curve fitting” because it was designed after observing that the true k has two peaks. However, the method does **not** rely on knowing the true solution, for the following reasons.

The basis is generic. The family $\{1, \sin^2(\pi x), \sin^2(2\pi x)\}$ is a standard set of smooth basis functions on $[0, 1]$. Using a truncated basis expansion to represent an unknown field is a well-established technique in inverse problems, independent of the specific true k . The three-term expansion can approximate a wide variety of continuous, positive functions – not just the particular k in the dataset.

The parameters are learned. The values of B, w_1, w_2 are not set by hand; they are initialized to generic values and trained from the same loss that drives the neural network. If the true k had only one peak, the optimizer could learn $w_2 \approx 0$ and the regulariser would not force a second peak.

The prior is soft. The regularisation is a penalty term, not a hard constraint. The PDE and data losses contribute to the gradient of k_ϕ and can move it away from k_{trig} if that improves the overall fit. The final predicted k is therefore a compromise between the prior and the physics, not a rigid adherence to the trigonometric shape.

The residual error is non-zero. If the regulariser were simply forcing the network to replicate the true k , the k error would be near zero. The fact that the final error is 8.51% shows that the network did **not** simply copy a pre-programmed answer – the regulariser only provided a “hint” that helped escape the ill-posed local minimum.

In a real application without test data, one would still choose a low-dimensional, smooth parameterisation as a prior. The specific choice of trigonometric functions is motivated by their smoothness, simplicity, and ability to represent hump-like features that are physically

plausible for a rod with varying stiffness. The number of basis functions can be increased if the data suggest more complexity, or reduced if a simpler model suffices. The approach is entirely transferable to other inverse problems.

A.5 Connection to Tikhonov Regularisation and Prior Work

The soft penalty term introduced in Problem A,

$$\mathcal{L}_{\text{reg}} = w_{\text{reg}} \frac{1}{N_r} \sum_{j=1}^{N_r} (k_{\phi}(x_j) - k_{\text{trig}}(x_j))^2, \quad (\text{A.3})$$

is a direct extension of **Tikhonov regularisation** [4]. In its classical form, Tikhonov regularisation stabilises an ill-posed inverse problem by penalising the deviation of the unknown field from a fixed reference k_{ref} :

$$\mathcal{L}_{\text{reg}}^{\text{Tikhonov}} = \lambda \|k - k_{\text{ref}}\|^2.$$

Here, the reference k_{ref} is usually a smooth function chosen a priori, and λ controls the strength of the penalty.

The formulation differs in one crucial aspect: the reference $k_{\text{trig}}(x) = B + w_1 \sin^2(2\pi x) + w_2 \sin^2(\pi x)$ is **not fixed**, but parameterised by trainable scalars B, w_1, w_2 . Thus the prior itself adapts to the data, allowing the regulariser to “capture the moment” – it nudges the network toward a two-peak structure without rigidly prescribing the peak heights, widths, or positions. The regularisation weight $w_{\text{reg}} = 100$ plays the role of the Tikhonov parameter λ .

A similar additive penalty strategy was employed by Haghighat et al. [1] in a PINN-based inversion framework for solid mechanics, where total-variation regularisation was used to enforce smoothness of the unknown parameter field. My work follows the same philosophy: incorporating a soft constraint that encodes physically plausible behaviour, while still allowing the PDE and data losses to dominate the final solution.

A.6 Summary

The trigonometric regularisation is not a shortcut to the answer; it is a principled method for incorporating prior knowledge about the expected smoothness and multi-modal nature of the unknown field. The basis functions are generic, the parameters are learned, and the penalty is soft. This combination breaks the ill-posedness of the inverse problem without requiring access to the true solution, making it a valid and generalisable technique within the PINN framework.